PySpark Hands-on Exercises

DataFrames, transformations, joins, window functions, lazy execution, caching, and partitioning | Google Colab

100,000 sales records	3 DataFrames	25 exercises	1 final challenge

Instructions: Run the setup and dataset-generation cells in Google Colab. Complete the exercises using the generated DataFrames. No CSV upload is needed. Solutions are intentionally excluded.

Step 1 | Install and initialize PySpark

```
!pip -q install pyspark
```

```
from pyspark.sql import SparkSession
from pyspark.sql import functions as F
from pyspark.sql.window import Window
from pyspark import StorageLevel
import time

spark = SparkSession.builder \
    .appName("PySparkPractice") \
    .getOrCreate()
```

Step 2 | Create product and store DataFrames

```
products_data = [
    ("P01", "Laptop", "Electronics", 60000),
    ("P02", "Monitor", "Electronics", 14000),
    ("P03", "Headphones", "Accessories", 3500),
    ("P04", "Printer", "Office", 17000),
    ("P05", "Router", "Networking", 4500),
    ("P06", "Tablet", "Electronics", 24000)
]

products_df = spark.createDataFrame(
    products_data,
    ["product_id", "product_name", "category", "list_price"]
)

stores_data = [
    ("S01", "Hyderabad", "South"),
    ("S02", "Bengaluru", "South"),
    ("S03", "Mumbai", "West"),
    ("S04", "Delhi", "North")
]

stores_df = spark.createDataFrame(
    stores_data, ["store_id", "city", "region"]
)

products_df.show()
stores_df.show()
```

Step 3 | Generate 100,000 sales records

The following two cells create a synthetic sales dataset with intentional nulls in **discount_pct** and **salesperson_id**. Run Cell A before Cell B.

Cell A - Create base transaction columns

```
sales_base_df = (
    spark.range(1, 100001)
    .withColumnRenamed("id", "order_id")
    .withColumn(
        "order_date",
        F.date_add(
            F.to_date(F.lit("2026-01-01")),
            (F.col("order_id") % 180).cast("int")
        )
    )
    .withColumn(
        "product_id",
        F.concat(
            F.lit("P"),
            F.lpad(
                ((F.col("order_id") % 6) + 1).cast("string"),
                2, "0"
            )
        )
    )
    .withColumn(
        "store_id",
        F.concat(
            F.lit("S"),
            F.lpad(
                ((F.floor(F.col("order_id") / 3) % 4) + 1)
                .cast("string"),
                2, "0"
            )
        )
    )
    .withColumn(
        "quantity",
        ((F.col("order_id") % 8) + 1).cast("int")
    )
    .withColumn(
        "discount_pct",
        F.when(
            F.col("order_id") % 19 == 0,
            F.lit(None).cast("int")
        ).otherwise(
            ((F.floor(F.col("order_id") / 5) % 4) * 5)
            .cast("int")
        )
    )
)
```

Step 3 | Generate sales records (continued)

Cell B - Add status and payment details, then join product prices

```
sales_df = (
    sales_base_df
    .withColumn(
        "order_status",
        F.when(F.col("order_id") % 11 == 0, "Cancelled")
         .when(F.col("order_id") % 7 == 0, "Returned")
         .otherwise("Completed")
    )
    .withColumn(
        "payment_method",
        F.when(F.col("order_id") % 3 == 0, "UPI")
         .when(F.col("order_id") % 3 == 1, "Card")
         .otherwise("Cash")
    )
    .withColumn(
        "salesperson_id",
        F.when(
            F.col("order_id") % 17 == 0,
            F.lit(None).cast("string")
        ).otherwise(
            F.concat(
                F.lit("EMP"),
                F.lpad(
                    ((F.col("order_id") % 12) + 1).cast("string"),
                    2, "0"
                )
            )
        )
    )
    .join(
        products_df.select(
            "product_id",
            F.col("list_price").alias("unit_price")
        ),
        "product_id", "left"
    )
    .select(
        "order_id", "order_date", "store_id",
        "product_id", "quantity", "unit_price",
        "discount_pct", "order_status",
        "payment_method", "salesperson_id"
    )
)

print("Total Records:", sales_df.count())
sales_df.printSchema()
sales_df.show(10, truncate=False)
```

Available DataFrames: `sales_df` (100,000 records), `products_df` (6 products), and `stores_df` (4 stores).

Practice Exercises

Complete the tasks in order. Use the three DataFrames created in the preceding cells. No solutions are included.

Part A | DataFrame transformations

1. Select order_id, product_id, quantity, and unit_price from the sales DataFrame.

2. Filter all completed transactions where quantity is greater than 5.

3. Create a column named gross_revenue by multiplying quantity and unit_price.

4. Rename order_status to transaction_status, then drop the payment_method column.

Part B | Aggregations and joins

5. Calculate the total quantity sold for each product.

6. Join sales with product master data and calculate total gross revenue by product category.

7. Join sales with store master data and calculate total transactions for each city.

8. Join all three DataFrames and find the top five products by gross revenue for completed orders.

Part C | Null handling and conditions

9. Count null values in discount_pct and salesperson_id.

10. Replace null discounts with zero using na.fill().

11. Remove records where salesperson_id is null using na.drop().

12. Create order_size using when() and otherwise(): Small for quantity 1-3, Medium for 4-6, and Large for 7 or more.

Part D | Window and date functions

13. Rank products by total gross revenue within each category using dense_rank().

14. Calculate running gross revenue by store, ordered by date and order ID.

15. Extract the month from order_date and calculate monthly sales revenue for completed orders.

Part E | Lazy execution

16. Create three consecutive transformations: filter(), withColumn(), and select(). Add a Python print() after each transformation. Trigger execution only at the end using show().

17. Create an aggregation by product_id. Use explain() to inspect the execution plan before calling collect().

Part F | Caching and persistence

18. Filter completed orders, call cache(), and execute count() twice. Measure the execution time of each action.

19. Persist transactions with quantity greater than 5 using StorageLevel.MEMORY_AND_DISK. Execute two actions, then release the persisted data using unpersist().

20. Cache an aggregated DataFrame, inspect its execution plan, and identify InMemoryTableScan or InMemoryRelation.

Part G | Repartition and coalesce

21. Check the existing partition count, then repartition the sales DataFrame into 8 partitions.

22. Display the number of records in each partition using spark_partition_id().

23. Repartition the sales DataFrame into 6 partitions based on store_id. Inspect which stores appear in each partition.

24. Create a DataFrame with 12 partitions, then reduce it to 4 using coalesce(). Compare its execution plan with repartition(4).

25. Save the sales data into 4 CSV part files using repartition(4) and write.csv().

Final Challenge

End-to-end PySpark data-processing pipeline

Build a single pipeline using **sales_df**, **products_df**, and **stores_df**. Complete the following steps:

1. Filter completed orders.

2. Replace null discounts with zero.

3. Join product and store details.

4. Calculate gross revenue and discounted revenue.

5. Group the data by region and product category.

6. Calculate total revenue and transaction count.

7. Rank categories by revenue within each region.

8. Cache the final aggregated DataFrame.

9. Display the results.

10. Write the output into 2 Parquet part files.

Learning objective: Construct a complete PySpark data-processing pipeline while demonstrating transformations, actions, lazy execution, caching, and partition management.